Data-Driven Optimization

선형모형 모델링 - 고급

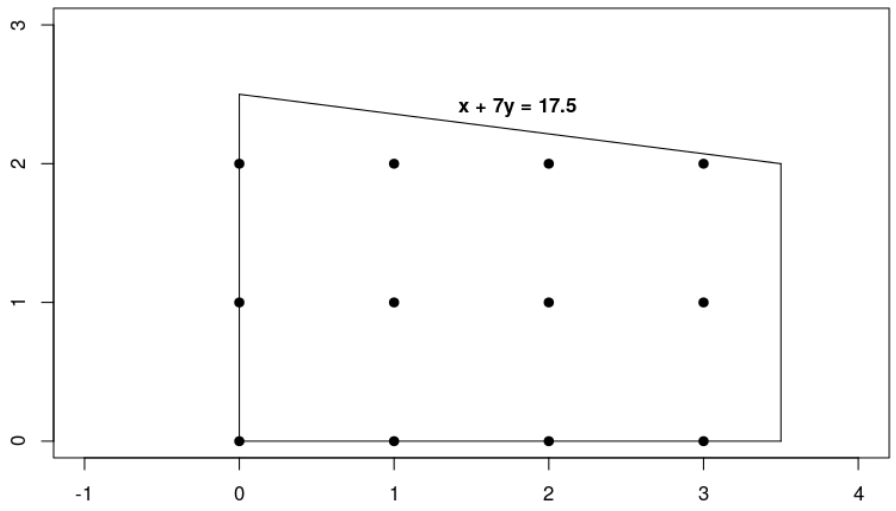
5주차

정수 계획법과 입치 최적화 모형

정수계획 모형 (Integer Programming Model)

정수계획법은 한정된 수의 가능해(feasible solution)을 가지고 있으므로 선형계획법보다 풀기가 쉽다(?).

maximize	$x + 10y$
s. t	$x + 7y \leq 17.5$
	$0 \leq x \leq 3.5$
	$0 \leq y$
	x, y는 정수



Binary Integer Programming Problem의 경우

- n = 10 → 1024 solutions
- n = 20 → over 1 million
- n = 30 → over 1 billion

정수계획 모형 (Integer Programming Model)

```
from ortools.linear_solver import pywraplp

# Create the mip solver with the SCIP backend.
solver = pywraplp.Solver.CreateSolver("SAT")

infinity = solver.infinity()
# x and y are integer non-negative variables.
#x = solver.IntVar(0.0, infinity, "x")
#y = solver.IntVar(0.0, infinity, "y")
x = solver.NumVar(0.0, infinity, "x")
y = solver.NumVar(0.0, infinity, "y")

solver.Add(x + 7 * y <= 17.5)
solver.Add(x <= 3.5)

solver.Maximize(x + 10 * y)

print(f"Solving with {solver.SolverVersion()}")
status = solver.Solve()
```

```
if status == pywraplp.Solver.OPTIMAL:
    print("Solution:")
    print("Objective value =", solver.Objective().Value())
    print("x =", x.solution_value())
    print("y =", y.solution_value())
else:
    print("The problem does not have an optimal solution.")
```

```
In [70]: runfile('C:/Users/1007/Dropbox/04_강의/2024년 1
wdir='C:/Users/1007/Dropbox/04_강의/2024년 1학기/00_데이
Solving with CP-SAT solver v9.8.3296
Solution:
Objective value = 23.0
x = 3.0
y = 2.0

In [71]: runcell(0, 'C:/Users/1007/Dropbox/04_강의/2024
or4-1.py')
Solving with CP-SAT solver v9.8.3296
Solution:
Objective value = 25.0
x = 0.0
y = 2.5
```

예제 1. 스태프 결정 문제

- 당신이 일주일에 7일 오픈하는 대중적인 Pluto dog의 핫도그 가판대를 운영한다고 가정하자. 당신은 5일을 일하고 2일을 연속해서 쉬는 종업원을 고용한다. 각 종업원은 동일한 주급을 받는다. 일주일 중의 어떤 날은 다른 날보다 바쁘다. 그리고 과거 경험에 의해 아래와 같이 특정 요일에 필요한 작업자의 수를 알고 있다.

요일	월	화	수	목	금	토	일
필요 스텝 수	20	16	13	16	19	14	12

종업원 요구 인원수를 만족하면서, 총 소요 종업원 수를 최소화하기 위하여 일주일의 각 요일에 몇 명의 종업원이 일을 시작해야 하는지를 결정하고자 한다.

예제 1. 스태프 결정 문제 – Modeling

X[i]: i 요일에 일을 시작하는 스태프의 수

Minimize

Obj: +1 x[0] +1 x[1] +1 x[2] +1 x[3] +1 x[4] +1 x[5] +1 x[6]

Subject to

MON: +1 x[0] +1 x[3] +1 x[4] +1 x[5] +1 x[6] >= 20

TUE: +1 x[0] +1 x[1] +1 x[4] +1 x[5] +1 x[6] >= 16

WED: +1 x[0] +1 x[1] +1 x[2] +1 x[5] +1 x[6] >= 13

THU: +1 x[0] +1 x[1] +1 x[2] +1 x[3] +1 x[6] >= 16

FRI: +1 x[0] +1 x[1] +1 x[2] +1 x[3] +1 x[4] >= 19

SAT: +1 x[1] +1 x[2] +1 x[3] +1 x[4] +1 x[5] >= 14

SUN: +1 x[2] +1 x[3] +1 x[4] +1 x[5] +1 x[6] >= 12

예제 1. 스태프 결정 문제 – Code

```
from ortools.linear_solver import pywraplp
solver = pywraplp.Solver.CreateSolver("SAT")

REQ = [20,16,13,16,19,14,12]
names = ['MON', 'TUE', 'WED', 'THU', 'FRI', 'SAT', 'SUN']
infinity = solver.infinity()

x = {}
for i in range(7):
    x[i] = solver.IntVar(0, infinity, "x[%i]" % i)

for i in range(7):
    constraint_expr = [x[(i-j) % 7] for j in range(5)]
    solver.Add(sum(constraint_expr) >= REQ[i], names[i])

objective = solver.Objective()
solver.Minimize(solver.Sum([x[i] for i in range(7)]))
```

```
# 모델 파일 생성
with open('or4-2.lp', "w") as out_f:
    lp_text = solver.ExportModelAsLpFormat(False)
    out_f.write(lp_text)

status = solver.Solve()

if status == pywraplp.Solver.OPTIMAL:
    print("Objective value =", solver.Objective().Value())
    for i in range(7):
        print(x[i].name(), " = ", x[i].solution_value())
else:
    print("The problem does not have an optimal solution.")
```

예제 2. 팀배정 문제

어떤 컨설팅 회사는 컨설턴트들을 2인 1조로 하여 고객 회사에 파견하여 경영혁신을 지원하고 있다. 과거의 업무 성과를 분석한 결과, 컨설턴트들이 특정 컨설턴트와 일을 할 때 더욱 성과가 좋다는 사실을 확인하였다. 이 회사는 8명의 컨설턴트가 있고 이 분석가들 사이에 팀워크 점수를 1(최악)에서 10(탁월)로 매긴 내용은 다음 표와 같다. 이 때 팀워크 점수가 최대가 되는 4개 조합을 찾는 이진 정수모형을 수립하고 그에 해당하는 프로그램을 작성하여라.

컨설턴트	1	2	3	4	5	6	7	8
1	-	9	3	4	2	1	5	6
2	-	-	1	7	3	5	2	1
3	-	-	-	4	4	2	9	2
4	-	-	-	-	1	5	5	2
5	-	-	-	-	-	8	7	6
6	-	-	-	-	-	-	2	3
7	-	-	-	-	-	-	-	4

예제 2. 팀배정 문제

- x_{ij} = 1 if 컨설턴트 i와 j가 한 쌍이 됨. (의사결정변수)
- R_{ij} : 분석가 i와 j의 팀워크 점수
- 목적함수 – 배정된 분석가들의 불일치 점수 합을 최소화

$$max \sum_{j<i} R_{ij} x_{ij}$$

- 제약식 1 – 모든 분석가는 다른 한 사람과 만 쌍을 이룸

$$x_{12}+x_{13}+x_{14}+x_{15}+x_{16}+x_{17}+x_{18}=1$$

$$x_{21}+x_{23}+x_{24}+x_{25}+x_{26}+x_{27}+x_{28}=1$$

$$x_{31}+x_{32}+x_{34}+x_{35}+x_{36}+x_{37}+x_{38}=1$$

$$\ldots$$

- 제약식 2 – 다음 두 변수는 같음. $x_{ij} = x_{ji}$ for 모든 i, j

예제 2. 팀배정 문제 - 모델링

Maximize

Obj: +9 X01 +3 X02 +4 X03 +2 X04 +1 X05 +5 X06 +6 X07 +1 X12 +7 X13 +3 X14 +5 X15 +2 X16 +1 X17 +4 X23 +4 X24 +2 X25 +9 X26 +2 X27 +1 X34 +5 X35 +5 X36 +2 X37 +8 X45 +7 X46 +6 X47 +2 X56 +3 X57 +4 X67

Subject to

consultant_0: +1 X01 +1 X02 +1 X03 +1 X04 +1 X05 +1 X06 +1 X07 = 1

consultant_1: +1 X10 +1 X12 +1 X13 +1 X14 +1 X15 +1 X16 +1 X17 = 1

consultant_2: +1 X20 +1 X21 +1 X23 +1 X24 +1 X25 +1 X26 +1 X27 = 1

consultant_3: +1 X30 +1 X31 +1 X32 +1 X34 +1 X35 +1 X36 +1 X37 = 1

consultant_4: +1 X40 +1 X41 +1 X42 +1 X43 +1 X45 +1 X46 +1 X47 = 1

consultant_5: +1 X50 +1 X51 +1 X52 +1 X53 +1 X54 +1 X56 +1 X57 = 1

consultant_6: +1 X60 +1 X61 +1 X62 +1 X63 +1 X64 +1 X65 +1 X67 = 1

consultant_7: +1 X70 +1 X71 +1 X72 +1 X73 +1 X74 +1 X75 +1 X76 = 1

x_0_1: +1 X01 -1 X10 = 0, x_0_2: +1 X02 -1 X20 = 0, x_0_3: +1 X03 -1 X30 = 0, x_0_4: +1 X04 -1 X40 = 0, x_0_5: +1 X05 -1 X50 = 0, x_0_6: +1 X06 -1 X60 = 0, x_0_7: +1 X07 -1 X70 = 0

x_1_2: +1 X12 -1 X21 = 0, x_1_3: +1 X13 -1 X31 = 0, x_1_4: +1 X14 -1 X41 = 0, x_1_5: +1 X15 -1 X51 = 0, x_1_6: +1 X16 -1 X61 = 0, x_1_7: +1 X17 -1 X71 = 0

x_2_3: +1 X23 -1 X32 = 0, x_2_4: +1 X24 -1 X42 = 0, x_2_5: +1 X25 -1 X52 = 0, x_2_6: +1 X26 -1 X62 = 0, x_2_7: +1 X27 -1 X72 = 0

x_3_4: +1 X34 -1 X43 = 0, x_3_5: +1 X35 -1 X53 = 0, x_3_6: +1 X36 -1 X63 = 0, x_3_7: +1 X37 -1 X73 = 0

x_4_5: +1 X45 -1 X54 = 0, x_4_6: +1 X46 -1 X64 = 0, x_4_7: +1 X47 -1 X74 = 0

x_5_6: +1 X56 -1 X65 = 0, x_5_7: +1 X57 -1 X75 = 0

x_6_7: +1 X67 -1 X76 = 0

예제 2. 팀배정 문제 – 모델링 (입력정보)

```
Ratings = [
        [None,9,3,4,2,1,5,6],
        [None,None,1,7,3,5,2,1],
        [None,None,None,4,4,2,9,2],
        [None,None,None,None,1,5,5,2],
        [None,None,None,None,None,8,7,6],
        [None,None,None,None,None,None,2,3],
        [None,None,None,None,None,None,None,4]
]
```

예제 2. 팀배정 문제 – 모델링 (의사결정 변수)

```
nC = len(Ratings[0])

# 의사결정 변수
X = {}
for i in range(nC):
    for j in range(nC):
        if i != j:
            X[i, j] = solver.IntVar(0, 1, "X"+str(i)+str(j))
```

예제 2. 팀배정 문제 – 모델링 (제약조건)

```
# 제약조건
const_expr = []
# 컨설턴트당 1명만 배정
for i in range(nC):
    const_expr = [X[i,j] for j in range(nC) if i != j]
    solver.Add(sum(const_expr) == 1, 'consultant_' + str(i))

# xij = xji 제약
for i in range(nC):
    for j in range(nC):
        if i < j:
            solver.Add(X[i, j] == X[j, i], 'x_'+str(i)+'_'+str(j))
```

예제 2. 팀배정 문제 – 모델링 (목적함수)

```
# 목적함수
obj_expr = []
for i in range(nC-1):
    for j in range(nC):
        if Ratings[i][j] != None:
            obj_expr.append(Ratings[i][j]*X[i, j])

solver.Maximize(solver.Sum(obj_expr))
```

예제 2. 팀배정 문제 - 결과출력

```
# 모델 파일 생성
with open('or4-4.lp', "w") as out_f:
    lp_text = solver.ExportModelAsLpFormat(False)
    out_f.write(lp_text)

status = solver.Solve()

if status == pywraplp.Solver.OPTIMAL:
    print("Objective value = %.1f" % solver.Objective().Value())
    for i in range(nC):
        for j in range(nC):
            if i != j and X[i, j].solution_value() != 0:
                print('Consultant [%i]' %i, '--> Consultant [%i]' %j)
else:
    print("The problem does not have an optimal solution.")
```

예제 3. 정수 계획 문제

- C 회사는 대구, 광주 중에 하나 또는 두 곳에 다 새로운 공장을 새우려고 한다. 또 제조품 저장을 위한 창고도 공장이 세워지는 장소에 한 개만 세우려고 한다. 각 투자에 대한 가치와 필요 자금은 다음과 같다.

번호	질문	의사결정 변수	투자의 현재 가치	소요 자금
1	대구에 공장을 세울 것인가?	x_1	90억	60억
2	광주에 공장을 세울 것인가?	x_2	50억	30억
3	대구에 창고를 지을 것인가?	x_3	60억	50억
4	광주에 창고를 지을 것인가?	x_4	40억	20억

- 보유자금 110억원

예제 3. 정수 계획 문제 – Modeling

- 의사결정 변수 $x_j = \begin{cases} 1, & \text{의사결정 } j\text{가 예이면} \\ 0, & \text{의사결정 } j\text{가 아니오이면} \end{cases} \quad j = 1,2,3,4$

- 목적함수

  Z = 투자의 현재 가치의 합

  $Z = 9x_1 + 5x_2 + 6x_3 + 4x_4$

- 제약조건

  $6x_1 + 3x_2 + 5x_3 + 2x_4 \leq 11$

  $x_3 + x_4 \leq 1$

  $x_3 \leq x_1$

  $x_4 \leq x_2$

  x_j is binary for $j = 1, 2, 3, 4$

예제 3. 정수 계획 문제 – coding

```
from ortools.linear_solver import pywraplp

solver = pywraplp.Solver.CreateSolver("SCIP")

nVars = 4
values = [9, 5, 6, 4]
reqs = [6, 3, 5, 2]

# 의사결정변수
x = {}
for i in range(nVars):
    x[i] = solver.IntVar(0, 1, "x[%i]" % i)

# 제약조건
solver.Add(sum([reqs[i]*x[i] for i in range(nVars)]) <= 11, 'const 0')
solver.Add(x[2] + x[3] <= 1, 'const 1')
solver.Add(x[2] <= x[0], 'const 2')
solver.Add(x[3] <= x[1], 'const 3')
```

```
# 목적함수
solver.Maximize(sum([values[i]*x[i] for i in range(nVars)]))

with open('or5-1.lp', "w") as out_f:
    lp_text = solver.ExportModelAsLpFormat(False)
    out_f.write(lp_text)

status = solver.Solve()

if status == pywraplp.Solver.OPTIMAL:
    print("OPTIMAL")
    print("목적함수값 = ", solver.Objective().Value())
    for i in range(nVars):
        print(x[i].name(), " = ", x[i].solution_value())
else:
    print("The problem does not have an optimal solution.")
```

정수 계획 문제 – Either-Or 제약조건

- big-M : 제약식의 의미가 없어질 정도로 큰 수 (예: 1,000,000)

- Either $3x_1 + 2x_2 \leq 18$ or $x_1 + 4x_2 \leq 16$ 제약식
  - A, B 2개의 기계 중 하나를 도입하고자 하는데 도입 결정을 포함해서 문제를 최적화하고자 한다면?

$$3x_1 + 2x_2 \leq 18 + M \cdot y$$
$$x_1 + 4x_2 \leq 16 + M \cdot (1-y)$$

y는 이진 변수

정수 계획 문제 – K out of N 제약조건

$$f_1(x_1, x_1, \dots, x_p) \le d_1$$

$$f_2(x_1, x_1, \dots, x_p) \le d_2$$

$$\dots$$

$$f_N(x_1, x_1, \dots, x_p) \le d_N$$

$$\Rightarrow$$

$$f_1(x_1, x_1, \dots, x_p) \le d_1 + M \cdot y_1$$

$$f_2(x_1, x_1, \dots, x_p) \le d_2 + M \cdot y_2$$

$$\dots$$

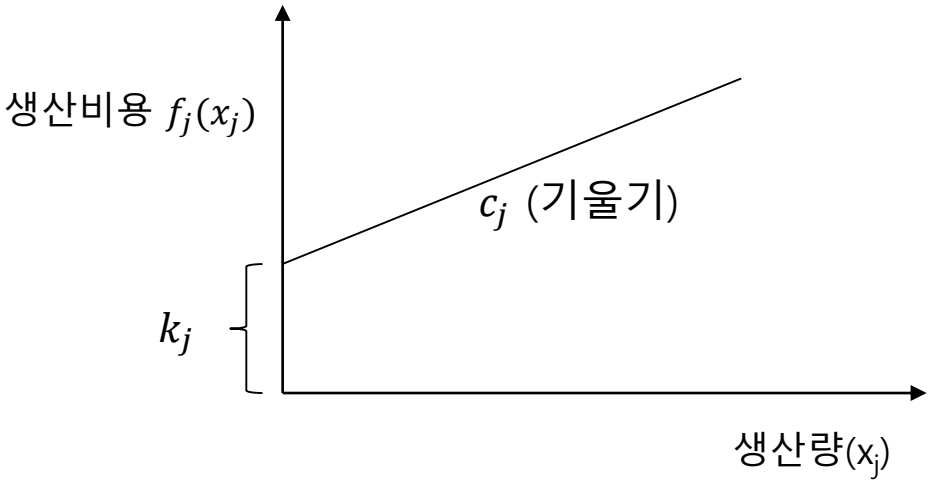
$$f_N(x_1, x_1, \dots, x_p) \le d_N + M \cdot y_N$$

N개 제약식 중에 K개 제약식이 만족되야 한다.

$$\sum_{i=1}^{N} y_i = N - K$$

정수 계획 문제 – 고정비용 문제 표현

- 생산비용 함수: $f_j(x_j) = \begin{cases} k_j + c_j x_j & if\ \ x_j > 0 \\ 0 & if\ \ x_j = 0 \end{cases} \quad j = 1,2,\dots,n$
  - k_j : j 기계의 고정비
  - c_j : j 기계의 변동비



- (목적함수) Minimize $Z = \sum_{j=1}^{n}(c_j x_j + k_j y_j)$ # n개 기계의 생산비용 최소화
- (제약조건) $x_j \le M \cdot y_j$ for j = 1, 2, …, n, y_j는 이진 변수

  + (기존 제약 조건)

예제 4. 정수 계획 문제 – 고정비용 문제

한 장난감 회사는 크리스마스 시즌을 대비하여 두 개의 장난감을 생산하고자 한다. 장난감 1과 2를 생산하기 위해서는 각각 $50,000과 $80,000의 셋 업 비용이 발생한다. 장난감 1과 2는 하나 당 $10과 $15의 이익이 발생한다. 회사는 두 개의 공장을 가지고 있고 이 두 공장은 모두 두 장난감을 생산할 수 있다. 그러나 운영상의 이유로 장난감을 둘 다 생산한다면 하나의 공장에서만 생산하려고 한다. 장난감 1은 공장 1에서 시간당 50개, 공장 2에서 시간당 40개가 생산된다. 그리고 장난감 2는 공장 1에서 시간당 40개, 공장 2에서 시간당 25개가 생산된다. 공장 1과 2는 크리스마스 전까지 500시간, 700시간의 생산이 가능하다. 이익을 최대화하기 위하여 각각 몇 개의 장난감 1, 2를 생산하여야 하는지 결정하여라.

예제 4. 정수 계획 문제 – 고정비용 문제 – Modeling

(의사결정변수)

- x_0, x_1는 장난감 1, 2의 생산량
- y_0는 0이면 공장 1 제약 활성화, 1이면 제약 2 활성화를 위한 이진변수
- y_1과 y_2은 장난감 1과 2의 고정비에 대한 이진변수

(제약조건)

- $\frac{x_0}{50} + \frac{x_1}{40} \leq 500 + M \cdot y_0$ # 공장 0 생산 제약
- $\frac{x_0}{40} + \frac{x_1}{25} \leq 700 + M \cdot (1 - y_0)$ # 공장 1 생산 제약
- $x_0 \leq BIGM \cdot y_1$
- $x_1 \leq BIGM \cdot y_2$

(목적함수)

Maximize $10x_0 + 15x_1 - 50{,}000y_1 - 80{,}000y_2$

예제 4. 정수 계획 문제 – 고정비용 문제 - Coding

```
from ortools.linear_solver import pywraplp

solver = pywraplp.Solver.CreateSolver("SCIP")
infinity = solver.infinity()
BIGM = 1000000

# 의사결정변수
x = {}
for i in range(2):
    x[i] = solver.IntVar(0, infinity, "x[%i]" % i)

y = {}
for i in range(3):
    y[i] = solver.IntVar(0, 1, "y[%i]" % i)

# 제약조건
solver.Add(x[0]/50 + x[1]/40 <= 500 + BIGM * y[0], 'const 0')
solver.Add(x[0]/40 + x[1]/25 <= 700 + BIGM * (1-y[0]), 'const 1')
solver.Add(x[0] <= BIGM * y[1], 'const 2')
solver.Add(x[1] <= BIGM * y[2], 'const 3')

# 목적함수
solver.Maximize(10 * x[0] + 15 * x[1] - 50000 * y[1] - 80000 * y[2])

with open('or5-1.lp', "w") as out_f:
    lp_text = solver.ExportModelAsLpFormat(False)
    out_f.write(lp_text)

status = solver.Solve()

if status == pywraplp.Solver.OPTIMAL:
    print("OPTIMAL")
    print("목적함수값 = ", solver.Objective().Value())
    for i in range(2):
        print(x[i].name(), " = ", x[i].solution_value())
    for i in range(3):
        print(y[i].name(), " = ", y[i].solution_value())
else:
    print("The problem does not have an optimal solution.")
```

실습

- 예제 $1-4$ 코드를 작성하고 실행하라.
- 다음 실습 $1-5$에 대한 모델링과 최적해를 구하는 코드를 작성하라.
- 모델링은 배포된 답안지에 작성하여 제출하시오.

실습 1. 스태프 결정 문제

온라인 쇼핑몰 허닭은 온라인 주문 포장을 위해 24시간 포장 작업을 하고 있는데 시간대별로 필요인력은 다음과 같다.

배치	시간	필요 작업자
1	12 a.m. ~ 6 a.m.	2
2	6 a.m. ~ 10 a.m.	8
3	10 a.m. ~ 12 p.m.	4
4	12 p.m. ~ 4 p.m.	3
5	4 p.m. ~ 6 p.m.	6
6	6 p.m. ~ 10 p. m.	5
7	10 p.m. ~ 12 a.m.	3

작업자는 4시간 동안 근무를 하고, 한 시간 동안 휴식을 취한 후, 추가로 4시간을 근무한다. 작업자는 시간 단위로 아무때나 작업을 시작할 수 있다고 하자. 고용하는 작업자 수를 최소화하기 위해 각 시간별로 몇 명의 작업자가 근무를 시작해야 하는 지 결정하여라.

의사결정변수 : X[i]: i 시간에 일을 시작하는 근무자의 수

실습 1. 스태프 결정 문제 2 – Modeling

X[i]: i 시간에 일을 시작하는 근무자의 수

Minimize

Obj: +1 x[0] +1 x[1] +1 x[2] +1 x[3] +1 x[4] +1 x[5] +1 x[6] +1 x[7] +1 x[8] +1 x[9] +1 x[10] +1 x[11] +1 x[12] +1 x[13] +1 x[14] +1 x[15] +1 x[16] +1 x[17] +1 x[18] +1 x[19] +1 x[20] +1 x[21] +1 x[22] +1 x[23]

Subject to

_0: +1 x[0] +1 x[16] +1 x[17] +1 x[18] +1 x[19] +1 x[21] +1 x[22] +1 x[23] >= 2

_1: +1 x[0] +1 x[1] +1 x[17] +1 x[18] +1 x[19] +1 x[20] +1 x[22] +1 x[23] >= 2

_2: +1 x[0] +1 x[1] +1 x[2] +1 x[18] +1 x[19] +1 x[20] +1 x[21] +1 x[23] >= 2

_3: +1 x[0] +1 x[1] +1 x[2] +1 x[3] +1 x[19] +1 x[20] +1 x[21] +1 x[22] >= 2

_4: +1 x[1] +1 x[2] +1 x[3] +1 x[4] +1 x[20] +1 x[21] +1 x[22] +1 x[23] >= 2

_5: +1 x[0] +1 x[2] +1 x[3] +1 x[4] +1 x[5] +1 x[21] +1 x[22] +1 x[23] >= 2

_6: +1 x[0] +1 x[1] +1 x[3] +1 x[4] +1 x[5] +1 x[6] +1 x[22] +1 x[23] >= 8

…

_20: +1 x[12] +1 x[13] +1 x[14] +1 x[15] +1 x[17] +1 x[18] +1 x[19] +1 x[20] >= 5

_21: +1 x[13] +1 x[14] +1 x[15] +1 x[16] +1 x[18] +1 x[19] +1 x[20] +1 x[21] >= 5

_22: +1 x[14] +1 x[15] +1 x[16] +1 x[17] +1 x[19] +1 x[20] +1 x[21] +1 x[22] >= 3

_23: +1 x[15] +1 x[16] +1 x[17] +1 x[18] +1 x[20] +1 x[21] +1 x[22] +1 x[23] >= 3

실습 1. 스태프 결정 문제 2 – Code

```
from ortools.linear_solver import pywraplp

solver = pywraplp.Solver.CreateSolver("SAT")

REQ = [2, 2, 2, 2, 2, 2, 8, 8, 8, 8,
       4, 4, 3, 3, 3, 3, 6, 6, 5, 5,
       5, 5, 3, 3]

infinity = solver.infinity()

x = {}
for i in range(24):
    x[i] = solver.IntVar(0, infinity, "x[%i]" % i)

objective = solver.Objective()
solver.Minimize(solver.Sum([x[i] for i in range(24)]))

# 모델 파일 생성
with open(‘or5-1.lp', "w") as out_f:
    lp_text = solver.ExportModelAsLpFormat(False)
    out_f.write(lp_text)

status = solver.Solve()

if status == pywraplp.Solver.OPTIMAL:
    print("Objective value =", solver.Objective().Value())
    for i in range(24):
        print(x[i].name(), " = ", x[i].solution_value())
else:
    print("The problem does not have an optimal solution.")
```

실습 2. 컨설턴스 배정 최적화

A 컨설팅 회사는 7명의 컨설턴트를 8개의 프로젝트 중 7개에 배정하려 한다. 각 컨설턴트는 정확히 1개의 프로젝트에만 배정되며, 각 프로젝트는 최대 1명의 컨설턴트가 담당한다. 아래 표는 각 컨설턴트가 각 프로젝트를 수행할 때 예상되는 비용(만 원)이다. 1 개 프로젝트는 진행하지 않음.

컨설턴트₩프로젝트	1	2	3	4	5	6	7	8
1	15	18	21	12	25	19	14	22
2	20	14	17	23	16	21	18	13
3	11	22	19	16	20	15	23	17
4	24	16	13	20	18	22	11	19
5	17	20	22	14	13	17	20	16
6	19	13	16	21	22	14	17	20
7	22	17	20	18	15	20	16	21

총 배정 비용을 최소화하는 배정 방안을 구하시오.

실습 2. 컨설턴스 배정 최적화 - 모델링

- 의사결정변수: $x_{ij} = 1$ 컨설턴트 i가 프로젝트 j에 배정되면

- 목적함수

$$\min \sum_{i=1}^{7} \sum_{j=1}^{8} c_{ij}\, x_{ij}$$

- 제약조건

$$\sum_{j=1}^{8} x_{ij} = 1 \qquad i = 1, \dots, 7$$

$$\sum_{j=1}^{7} x_{ij} = 1 \qquad i = 1, \dots, 8$$

컨설턴트	배정 프로젝트	비용
1	프로젝트 4	12
2	프로젝트 8	13
3	프로젝트 1	11
4	프로젝트 7	11
5	프로젝트 5	13
6	프로젝트 2	13
7	프로젝트 3	20
(미배정)	프로젝트 6	-

총 최소 비용 = 93

실습 3. 생산계획 문제 1

- 경영자를 위한 전용기를 제작 판매하는 A사는 최근 3 군데 고객으로 부터 다음 표와 같은 구매 요청을 받았다. 현재 생산 중에 있는 제품과 짧은 납품 일정을 고려할 때 3 요청을 모두 받아 들일 수 없는 상황이다.

	고객		
	1	2	3
초기비용	$3M	$2M	0
이익 (주문 당)	$2M	$3M	$0.8M
가용한 생산 용량 중 생산에 필요한 용량 (주문 당)	20%	40%	20%
최대 주문 수	3	2	5

- A는 이익을 최대화하기 위하여 각 고객별로 몇 대의 비행기를 제작하여야 하는가?

실습 3. 생산계획 문제 1

x_i 는 i번째 고객의 제품을 생산량, 정수, i = 1, 2, 3
y_i 는 i번째 고객의 제품을 생산하면 1, 아니면 0인 이진 정수, I = 1, 2, 3
M = 100,000,000

$$
\begin{aligned}
\text{maximize} \quad & 2x_1 + 3x_2 + 0.8x_3 - 3y_1 - 2y_2 \\
\text{subject to} \quad & 0.2x_1 + 0.4x_2 + 0.2x_3 \leq 1 \\
& x_1 \leq M \cdot y_1 \\
& x_2 \leq M \cdot y_2 \\
& x_1 \leq 3 \\
& x_2 \leq 2 \\
& x_3 \leq 5
\end{aligned}
$$

$x_1, x_2, x_3 \geq 0$ 인 정수, y_1, y_2는 이진 변수

실습 4. 생산계획 문제 2

- 프로그레시브사는 4개 제품에 대한 생산라인을 운영하고있다. 경영층은 어떤 생산 라인을 어떤 생산 수준에서 운영할 것인지 결정하고자 한다. 셋업 비용과 단위 제품당 수익이 다음과 같을 때 총 수익들 최대화하는 product mix를 구하여라.

	Product			
	1	2	3	4
Start-up cost	$50,000	$40,000	$70,000	$60,000
Marginal revenue	$ 70	$ 60	$ 90	$ 80

- x_1, x_2, x_3, x_4를 제품 1, 2, 3, 4의 제품 생산 수준이라 하고 다음 제약을 고려하여 최적 생산 계획을 위한 최적화 정수모형과 Python 프로그램을 작성하여라.

(1) 2개까지의 신제품만 생산할 수 있다.

(2) 제품 1 또는 2가 생산되어야 제품 3 또는 4가 생산될 수 있다.

(3) 제품 생산을 위한 공통재료는 6000까지 활용 가능하고 다음 2가지 제품별 사용량을 갖는다.
Either $5x_1 + 3x_2 + 6x_3 + 4x_4 \leq 6000$ or $4x_1 + 6x_2 + 3x_3 + 5x_4 \leq 6000$

실습 4. 생산계획 문제 2 - 모델링

- maximize $70x_1 - 50{,}000y_1 + 60x_2 - 40{,}000y_2 + 90x_3 - 70{,}000y_3 + 80x_4 - 60{,}000y_4$

  subject to $y_1 + y_2 + y_3 + y_4 \leq 2$

  $y_3 \leq y_1 + y_2$

  $y_4 \leq y_1 + y_2$

  $5x_1 + 3x_2 + 6x_3 + 4x_4 \leq 6000 + M \cdot y_5$

  $4x_1 + 6x_2 + 3x_3 + 5x_4 \leq 6000 + M \cdot (1 - y_5)$

  $x_1 \leq M \cdot y_1,\ x_2 \leq M \cdot y_2,\ x_3 \leq M \cdot y_3,\ x_4 \leq M \cdot y_4$

x_i 는 정수, i = 1, 2, 3, 4, y_i 는 이진 정수, i = 1, 2, 3, 4,5

M = 100,000,000

	Product			
	1	2	3	4
Start-up cost	$50,000	$40,000	$70,000	$60,000
Marginal revenue	$ 70	$ 60	$ 90	$ 80

실습 5. 할당문제

은퇴한 미국인들이 따뜻한 기후로 이주하는 경우가 점점 늘어나고 있다. 이 추세를 활용하기 위해 Sunny Skies Unlimited는 대규모 부동산 개발 프로젝트를 추진 중이다. 이 프로젝트는 완전히 새로운 은퇴 커뮤니티인 'Pilgrim Haven'을 수 제곱 마일의 크기의 장소에 조성하는 것이다. 주요 의사결정 중 하나는 커뮤니티에 배정된 두 개의 소방서를 어디에 설치할지 결정하는 것이다. 계획에 따라 Pilgrim Haven을 다섯 개 구역(tract)으로 나누고, 각 구역에는 소방서를 최대 하나를 설치한다. 각 소방서는 자신이 설치된 구역에서 발생한 화재 뿐만 아니라, 자신에게 할당된 다른 구역의 화재에도 대응해야 한다. 따라서 의사결정은 (1) 어떤 구역에 소방서를 설치할 것인지와 (2) 나머지 각 구역을 어느 소방서에서 대응할 것인지이다. 목표는 화재 발생 시 전반적인 평균 대응 시간을 최소화하는 것이다. 아래 표는 각 구역(열)에 화재가 발생했을 때, 해당 구역이 특정 구역(행)에 위치한 소방서로부터 서비스를 받을 경우의 평균 대응 시간을 나타낸다. 표의 마지막 행은 각 구역에서 하루 평균 발생이 예상되는 화재 수를 보여준다.

이 문제에 대한 이진정수 모형을 수립하고 파이썬 코딩으로 해를 찾아라.

소방서가 설치된 구역	해당 구역에서의 화재에 대한 대응시간 (분)				
	1	**2**	**3**	**4**	**5**
1	5	12	30	20	15
2	20	4	15	10	25
3	15	20	6	15	12
4	25	15	25	4	10
5	10	25	15	12	5
1일 평균 화재 횟수	2	1	3	1	3

실습 5. 할당문제

Minimize

Obj: +10 y[0][0] +12 y[0][1] +90 y[0][2] +20 y[0][3] +45 y[0][4] +40 y[1][0] +4 y[1][1] +45 y[1][2] +10 y[1][3] +75 y[1][4] +30 y[2][0] +20 y[2][1] +18 y[2][2] +15 y[2][3] +36 y[2][4] +50 y[3][0] +15 y[3][1] +75 y[3][2] +4 y[3][3] +30 y[3][4] +20 y[4][0] +25 y[4][1] +45 y[4][2] +12 y[4][3] +15 y[4][4]

Subject to

two_station: +1 x[0] +1 x[1] +1 x[2] +1 x[3] +1 x[4] = 2

assign_st_0: +1 y[0][0] +1 y[1][0] +1 y[2][0] +1 y[3][0] +1 y[4][0] = 1

assign_st_1: +1 y[0][1] +1 y[1][1] +1 y[2][1] +1 y[3][1] +1 y[4][1] = 1

assign_st_2: +1 y[0][2] +1 y[1][2] +1 y[2][2] +1 y[3][2] +1 y[4][2] = 1

assign_st_3: +1 y[0][3] +1 y[1][3] +1 y[2][3] +1 y[3][3] +1 y[4][3] = 1

assign_st_4: +1 y[0][4] +1 y[1][4] +1 y[2][4] +1 y[3][4] +1 y[4][4] = 1

st배치보장_0_0: -1 x[0] +1 y[0][0] <= 0

st배치보장_0_1: -1 x[0] +1 y[0][1] <= 0

st배치보장_0_2: -1 x[0] +1 y[0][2] <= 0

st배치보장_0_3: -1 x[0] +1 y[0][3] <= 0

st배치보장_0_4: -1 x[0] +1 y[0][4] <= 0

st배치보장_1_0: -1 x[1] +1 y[1][0] <= 0

st배치보장_1_1: -1 x[1] +1 y[1][1] <= 0

st배치보장_1_2: -1 x[1] +1 y[1][2] <= 0

st배치보장_1_3: -1 x[1] +1 y[1][3] <= 0

st배치보장_1_4: -1 x[1] +1 y[1][4] <= 0

st배치보장_2_0: -1 x[2] +1 y[2][0] <= 0

st배치보장_2_1: -1 x[2] +1 y[2][1] <= 0

st배치보장_2_2: -1 x[2] +1 y[2][2] <= 0

st배치보장_2_3: -1 x[2] +1 y[2][3] <= 0

st배치보장_2_4: -1 x[2] +1 y[2][4] <= 0

st배치보장_3_0: -1 x[3] +1 y[3][0] <= 0

st배치보장_3_1: -1 x[3] +1 y[3][1] <= 0

st배치보장_3_2: -1 x[3] +1 y[3][2] <= 0

st배치보장_3_3: -1 x[3] +1 y[3][3] <= 0

st배치보장_3_4: -1 x[3] +1 y[3][4] <= 0

st배치보장_4_0: -1 x[4] +1 y[4][0] <= 0

st배치보장_4_1: -1 x[4] +1 y[4][1] <= 0

st배치보장_4_2: -1 x[4] +1 y[4][2] <= 0

st배치보장_4_3: -1 x[4] +1 y[4][3] <= 0

st배치보장_4_4: -1 x[4] +1 y[4][4] <= 0